Teach LLMs to Personalize – An Approach inspired by Writing Education

Cheng Li
Google
USA
chgli@google.com

Mingyang Zhang
Google
USA
mingyang@google.com

Qiaozhu Mei*
University of Michigan
USA
qmei@umich.edu

Yaqing Wang
Google
USA
yaqingwang@google.com

Spurthi Amba Hombaiah
Google
USA
spurthiah@google.com

Yi Liang
Google
USA
yiliang@google.com

Michael Bendersky
Google
USA
bemike@google.com

arXiv:2308.07968v1 [cs.CL] 15 Aug 2023

ABSTRACT

Personalized text generation is an emerging research area that has attracted much attention in recent years. Most studies in this direction focus on a particular domain by designing bespoke features or models. In this work, we propose a general approach for personalized text generation using large language models (LLMs). Inspired by the practice of writing education, we develop a multistage and multitask framework to teach LLMs for personalized generation. In writing instruction, the task of writing from sources is often decomposed into multiple steps that involve finding, evaluating, summarizing, synthesizing, and integrating information. Analogously, our approach to personalized text generation consists of multiple stages: retrieval, ranking, summarization, synthesis, and generation. In addition, we introduce a multitask setting that helps the model improve its generation ability further, which is inspired by the observation in education that a student's reading proficiency and writing ability are often correlated. We evaluate our approach on three public datasets, each of which covers a different and representative domain. Our results show significant improvements over a variety of baselines.

KEYWORDS

personalized generation, large language models

*Work done as a visiting researcher at Google.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
XXX, XXX, XXX
© 2023 Association for Computing Machinery.
ACM ISBN 978-1-4503-XXXX-X/18/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Cheng Li, Mingyang Zhang, Qiaozhu Mei, Yaqing Wang, Spurthi Amba Hombaiah, Yi Liang, and Michael Bendersky. 2023. Teach LLMs to Personalize – An Approach inspired by Writing Education. In *Proceedings of XXX*. ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

As artificial intelligence (AI) based systems are increasingly used to assist content creation, there has been a tremendous amount of interest in personalized text generation. Producing a customized response that takes into account auxiliary context, such as documents previously written by the user, is crucial for the development of generative systems that support specific audiences, creation contexts, and information needs. Example applications include AI-assisted writing of various types of content from tweets and news stories to scientific articles and fictions, corporate and personal communications (emails, chats, forums), and transformations of a given piece of written content into other styles, e.g., summarization, or conversely, elaboration.

Researchers have investigated the generation of personalized text on various domains, including but not limited to reviews [13, 14], dialogue agents [17, 38, 41] and social networks [8]. Previous work mostly relies on domain-specific features or knowledge and proposes models that address a particular task. How to design a general approach that works for all scenarios is a less studied area.

On the other hand, along with the ascendance of generative AI, through chatbots like ChatGPT[1] and Bard[2] in particular, large language models (LLMs) are playing an increasingly prominent role in many text generation tasks. However, few studies have explored how to equip LLMs with the ability to personalize.

In this work, we propose a general approach for personalized text generation using large language models. Our work is inspired by the widely-used practice in writing education, which decomposes the task of writing from sources by a procedure of finding, evaluating, summarizing, synthesizing, and integrating information [3, 31].

[1]https://chat.openai.com
[2]https://bard.google.com

Analogously, we adopt a multistage multitask framework to teach LLMs for personalized text generation, with similar stages being retrieval, ranking, summarization, synthesis, and generation. Specifically, given the immediate context, such as the title and the starting sentence of a document a user is writing, we formulate a query and *retrieve* relevant information from an auxiliary repository of personal contexts, such as documents the user has authored in the past. We then *rank* the retrieved results based on their relevance and importance, followed by *summarizing* the ranked results. We also *synthesize* the retrieved information into key elements, and finally feed the retrieved results, summary and synthesized information into the large language model for generating the new document.

In language education, it is often observed that the proficiency of one's writing skills is highly correlated with that of their reading skills [4]. Furthermore, studies show that author recognition tasks can be used to measure the amount and level of reading by an individual [18], which correlates with their reading proficiency. Inspired by these two observations, we create a multitask setting that aims to improve the reading ability of the large language model, where we introduce an auxiliary task charging the model to attribute the authorship of a given text. We anticipate that this task will help the model better understand (i.e., read) the given text and in turn generate (i.e., write) better and more personalized content.

We evaluate the proposed models on three public datasets, which cover personal email communications, social media discussions, and product reviews. By employing our multistage and multitask framework, we demonstrate significant improvements over a variety of baselines on all three datasets.

2 RELATED WORK

We present a literature review on personalized text generation and two related tasks, controlled text generation and text style transfer.

Personalized text generation. Some studies focus on improving personalized generation for a particular domain by utilizing domain-specific features or knowledge. Li and Tuzhilin [14] design a model based on self-attentive recursive autoencoders to generate personalized user reviews given product description, sentiment labels, and historical reviews of the user. A knowledge enhanced personalized review generation model based on a capsule graph neural network (CapsGNN) is proposed in [13] to utilize product attributes. Gao et al. [8] focus on personalized social text generation, where personalized features are fed to the encoder to guide the generation of the decoder. There are extensive studies on personalization for dialogue agents [17, 38, 41]. Due to limited real conversational data, researchers have explored constructing data by asking crowd-workers to write dialogues for specific personas [41] and by extracting user attributes and utterances from Reddit [17, 38] and Weibo [25, 43].

There are investigations on using predefined attributes and topics for personalization. A personalized sentence generation method is proposed [40] based on generative adversarial networks (GANs). Frequently used function words and content words are used as input and as sentence structure constraints for model training.

A less explored area is how to utilize large language models for personalized generation across different domains without relying on domain-specific or user-defined features. LaMP [29] is the work closest to ours. It provides a benchmark for training and evaluating personalized language models on three classification and four text generation tasks. They deploy an approach that retrieves text from user profiles. The generation tasks provided in LaMP are at the sentence-level. We instead consider generating longer text of passage-length, which is more challenging. Method-wise, the retrieval based approach in LaMP can be viewed as an instantiation of a single component of the multi-stage framework we proposed. Skopyk et al. [34] propose to train transformer layer adapters to achieve the effect of personalization. The paper only proposes the method without including any experimental analysis.

Controlled text generation. Controlled text generation aims to generate text with a predefined list of attributes, which could be stylistic or semantic. To reduce the cost of finetuning, recent work of controlled text generation resorts to decoding-time methods, directly making pre-trained models generate texts towards desired attributes during inference. These methods include PPLM [5], GeDi [11], FUDGE [39], and DEXPERTS [16].

Controlled text generation is different from personalized generation in that it requires a predefined set of attributes (constraints).

Text style transfer. A task related to controlled text generation is text style transfer. Its goal is to transform a piece of text by controlling certain attributes of the generated text while preserving the content. There are two paradigms: supervised learning using parallel corpora, and unsupervised methods using non-parallel corpora. With parallel data, standard sequence-to-sequence models can be directly employed [27]. There are three approaches when only non-parallel corpora are available. The first approach is to disentangle text into content and attributes for generative modeling [33]. The second approach, called prototype editing [12], extracts a sentence template and attribute markers for generation. The third approach constructs pseudo-parallel corpora to train the model [42].

Unlike text style transfer, personalized generation does not assume that the original text is already given. Like controlled text generation, most methods for text style transfer expect a given set of predefined attributes, which are not available in our setting.

Our work is also aligned with the paradigm of teaching LLMs to reason through chain of thoughts [36, 37], with our decomposition of tasks deeply inspired by writing education and our model training for each task going beyond prompt engineering.

3 PROBLEM FORMULATION

We consider the setting where a user is writing a document, which we call the **current document**. Given the **immediate context** and the user's **personal context**, the goal of the personalized model is to complete the document so that the generated document is close to the real current document as if the user finishes writing.

There might be different ways to define the immediate context and the personal context. For simplicity and generality, we use the title and a short start of the current document as the immediate context. The user's personal context is defined as the documents they have written in the past at the time of writing the current document. These contexts are analogous to the sources a student is instructed to write from [3].

Our training task is formulated as follows. Given a list of examples $\{(x_{ut}, \mathcal{D}_{ut}, d_{ut})\}$, where x_{ut} is the immediate context of

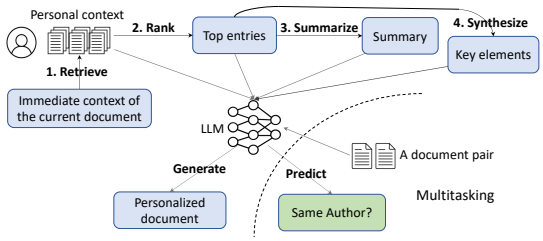

Figure 1: The overview of the multistage multitask framework for personalized text generation.

user u for the current document d_{ut} at time step t, and $\mathcal{D}_{ut} = \{d_{u1}, d_{u2}, ..., d_{u,t-1}\}$ is the personal context of past documents, we want to train a personalized generation model **G** that generates d'_{ut} based on $(x_{ut}, \mathcal{D}_{ut})$ so that we can maximize the similarity between d'_{ut} and d_{ut}. Whenever it is clear from the context, we will omit the subscript u and directly use x_t, $\mathcal{D}_t$ and d_t instead. We will also use "user" and "author" interchangeably.

4 METHOD OVERVIEW

The overview of our multistage multitask framework for personalized text generation is presented in Figure 1.

Given the immediate context x_t of the current document written by a user u, a retriever $\mathbf{Re}(x_t, \mathcal{D}_t)$ retrieves entries from past user document set $\mathcal{D}_t$ using x_t as the query. The returned entries are fed to a ranker **Ra** to produce ranked entries $\mathcal{E}_t = \mathbf{Ra}(\mathbf{Re}(x_t, \mathcal{D}_t))$, which are consumed by: (1) a summarization model $\mathbf{Su}(x_t, \mathcal{E}_t)$ to produce a summary of the multiple documents retrieved; and (2) a synthesis model $\mathbf{Sy}(x_t, \mathcal{E}_t)$ to produce key elements in these documents. The personalized generation model **G** generates the current document $d'_t = \mathbf{G}(x_t, \mathbf{Su}(x_t, \mathcal{E}_t), \mathbf{Sy}(x_t, \mathcal{E}_t), \mathcal{E}_t)$ and is trained against the ground-truth current document d_t.

We additionally consider an auxiliary task, called author distinction, to help the model better understand the user context and generate better personalized content. Given a document d_{ui} written by a user u, we randomly sample another document d_{vj} to form a document pair. The model **G** is then trained on a set of tuples $\{(d_{ui}, d_{vj}), y\}$, where the label $y = true$ if $v = u$, otherwise $y = false$. Note that we use text $\{true, false\}$ instead of numerical labels for y since **G** is a sequence-to-sequence model.

5 PERSONALIZED TEXT GENERATION

We discuss the detail of each stage as outlined in Section 4.

5.1 Retrieval

In the retrieval stage, given the immediate context x_t, the retriever $\mathbf{Re}(x_t, \mathcal{D}_t)$ uses x_t as the query to retrieve relevant text entries from past document set $\mathcal{D}_t$.

To define an immediate context that can be applied to any scenario, we simply use $FirstKCharacters(d_t)$, where $FirstKCharacters(\cdot)$ returns the first K characters of a piece of text. We set $K = 150$ for all experiments. If a document has a title, we concatenate the title and the body as the text.

We experiment with both sparse and dense retrievers to retrieve relevant entries from a user's past documents. We employ BM25 [28] as the sparse retriever. We use a T5X Retrieval model [19, 21], GTR-Large, as our dense retriever. We do not choose models of larger sizes since they demonstrate similar performance but much worse effectiveness-latency trade-offs on benchmark datasets [21].

For dense retrieval, we experiment with two levels of granularity when indexing personal document entries: a *document* level and a *snippet* level. We do not choose a sentence level since many sentences are too short to offer enough context information. We create a snippet in this way: we keep appending sentences from the same document until we reach 250 characters or we reach the end of the document.

Since the snippets to retrieve are quite short, we only examine the performance of sparse retrieval at the document level.

5.2 Ranking

Since we experiment with indexing entries at both document and snippet level, we can rank entries accordingly:

- RankDocBM25. For sparse retrieval, we retrieve and rank documents based on BM25 scores.
- RankDocDense. For dense retrieval, when we retrieve entries at the document level, we rank retrieved documents based on their embedding similarity with the embedding of the immediate context x_t.
- RankSnippet. Similarly, for dense retrieval, when we retrieve entries at the snippet level, we rank retrieved snippets based on embedding similarity.

During analysis, we find that issues exist for both RankDocDense and RankSnippet. The retrieved results via RankDocDense can be less relevant since embeddings are less effective when the documents to encode are long. While for RankSnippet, many similar snippets are retrieved, providing insufficient information for generation. For example, if the immediate context is *I really enjoyed reading the book*, we might retrieve similar snippets like *I enjoy the book*, *The book is fun*, or *I love this book*. They are all relevant but do not provide enough details on why this user enjoys a particular book, which is critical for passage-level generation.

To alleviate the two issues, we propose another dense retrieval strategy, RankDocBySnpt, inspired by past work on using passage evidence in retrieval [2]. RankDocBySnpt retrieves relevant text at the snippet level, which addresses the issue that document embeddings are less effective. At the ranking stage, instead of directly ranking snippets, we rank documents that contain the retrieved snippets, to mitigate lack of diversity in snippets retrieved via RankSnippet. Specifically for each document d_i, we compute the embedding similarity score between each retrieved snippet $s_{ij} \in d_i$ and the immediate context x_t, and use the max score as the document score for ranking. That is, $score(d_i, x_t) = \max_{s_{ij} \in d_i}(score(s_{ij}, x_t))$.

To make all the ranking strategies comparable, we concatenate ranked entries into a string and truncate it to 2,500 characters. Thus the subsequent modules are fed with input text of the same length.

5.3 Summarization

The summarization stage aims to extract important information from the retrieved entries so that the generation model can have a better understanding of what is the most important information in the user's personal context, such as key points, topics, or useful

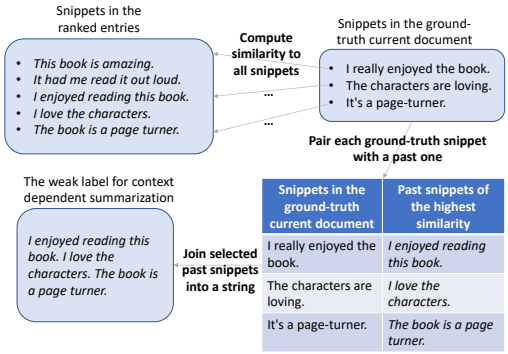

Figure 2: Creation of weak labels for context dependent summarization.

phrases (so they can be reflected in the output). With the summary, the generation model does not need to work on extracting the high-level aspects and generating the exact words at the same time, making the generation task easier.

We experiment with two strategies – context independent and context dependent summarization. By context dependent, we mean that the summarization is conditioned on the immediate context.

Context independent summarization. We choose a straightforward implementation of context independent summarization – we finetune an independent LLM, T5-11B [26], on publicly available summarization datasets, and directly use the finetuned model on our ranked entries for inference. The datasets we use are CNN/Daily Mail [30], ForumSum [9], and Reddit TIFU-long [10].

Context dependent summarization. The challenge to train a context dependent summarization model is the lack of ground-truth labels. We tackle this challenge by generating weak labels based on ground-truth current documents. Our intuition is that we want to extract text from the retrieved results that are more likely to be used in the current document. To this end, we find text from the ranked entries that is similar to the text of the current document, which can be formulated as an extractive summarization task. An example to illustrate this idea can be found in Figure 2.

Specifically for each snippet $s_{ti} \in d_t$ in the ground-truth current document, we compute its similarity to all snippets in the ranked entries $\mathcal{E}_t = \{e_1, e_2, ..., e_R\}$ retrieved from past documents, where e_j is the j-th snippet in the entries $\mathcal{E}_t$, and R is the number of snippets we include in the ranking. For simplicity, we reuse the embeddings obtained by the T5X Retrieval model [19, 21] in the retrieval stage to compute similarity. The snippet with the highest similarity score $e_{max_i} = \arg\max_{e_j \in \mathcal{E}_t}(score(e_j, s_{ti}))$ will be added to the candidate snippet list $\mathcal{L}_t$. If the selected snippet e_{max_i} is already in the candidate list $\mathcal{L}_t$, we look for the snippet with the next highest score until we find a new snippet. We keep adding newly selected snippets from the ranked entries to the candidate list until we have iterated through all the snippet pairs.

The weak labels are created by $Join(\mathcal{L}_t)$, which joins the snippets in the candidate list into one string. Given the immediate context x_t and the ranked entries $\mathcal{E}_t$, the context dependent summarization model $\mathbf{Su}(x_t, \mathcal{E}_t)$ is trained using the weak labels $Join(\mathcal{L}_t)$ to minimize the cross-entropy loss. We still choose T5-11B [26] as the model.

One important note is that we only use half of the training data for the generation task to train the summarization model. In this way, the generated summary will be noisier on the unseen half of the training data (similar to the test set). Consequently the generation model will be aware of the noisy summary during training and learn to cope with it by attending to other information.

Another note is that no validation or test data for the generation task are used to train the summarization model.

5.4 Synthesis

The synthesis step aims to find key elements that are common in the top retrieved entries for an overall understanding of the current writing task. We experiment with extracting keywords as the synthesis step in this paper. More sophisticated approaches to synthesis are worth exploration and are left as future work.

Similar to summarization, we investigate two strategies – context independent and context dependent synthesis. The context also refers to the immediate context.

Context independent synthesis. We extract keywords by finding frequent terms from the past documents $\mathcal{D}_t$. We limit terms to unigrams as most users do not have enough documents to extract n-grams with larger n. We also remove stopwords, words with frequency of one, and words with small inverse document frequency (IDF < 1.5). We then sort remaining words in descending order by their frequency and then by IDF, and keep up to 20 words.

Context dependent synthesis. Our idea of creating weak labels for context dependent synthesis is very similar to how we create weak labels for context dependent summarization – we aim to find important words from the retrieved results that are likely to be used in the generation of the current document.

To be more specific, for each source word $w_{ti} \in d_t$ in the ground-truth current document, we compute its similarity with each target word $v_{tj} \in \mathcal{E}_t$ from the ranked entries. For both source and target words, we skip stopwords or words with IDF < 1.5. Two words (w_{ti}, v_{tj}) are similar if at least one of the conditions are met:

- The two words are identical.
- The two words are synonyms as defined in WordNet [6].
- The two words are close in the embedding space. We use the uncased GloVe [24] embeddings pretrained on the Common Crawl dataset. We define two words as similar if their Euclidean distance is less than 4.

We add the qualified target word v_{tj} to the candidate target word list $\mathcal{T}_t$. After going through all possible (source, target) word pairs, the words in the candidate list $\mathcal{T}_t$ are then sorted inversely by the number of times they are selected, and then by IDF. The candidate word list is then joined into a string to form the weak label $Join(\mathcal{T}_t)$.

We still finetune a T5-11B [26] model for synthesis. Given the immediate context x_t and the ranked entries $\mathcal{E}_t$, the context dependent synthesis model $\mathbf{Sy}(x_t, \mathcal{E}_t)$ is trained using the weak label $Join(\mathcal{T}_t)$ to minimize the cross-entropy loss.

We use the same set of training examples that are used for summarization. The only difference is that the label is changed from the joined snippet list $Join(\mathcal{L}_t)$ to the joined target word list $Join(\mathcal{T}_t)$.

5.5 Personalized Generation

Given the immediate context x_t, the ranked entries $\mathcal{E}_t$ retrieved from the past documents, the context independent/dependent summary from the summarization model $\mathbf{Su}(x_t, \mathcal{E}_t)$, the context independent/dependent synthesis from the synthesis model $\mathbf{Sy}(x_t, \mathcal{E}_t)$, the personalized generation model $\mathbf{G}(x_t, \mathbf{Su}(x_t, \mathcal{E}_t), \mathbf{Sy}(x_t, \mathcal{E}_t), \mathcal{E}_t)$ is trained using the ground-truth current document d_t as the label to minimize the cross-entropy loss.

To help the model distinguish between various information sources, we add different prefixes when converting the sources to a single string input. Specifically, the immediate context is prefixed by *passage start*, the summary is prefixed by *summary*, the synthesis is prefixed by *important words*, and the list of ranked entries is prefixed by *past passages*.

5.6 Multitask Learning

Studies in language education show that writing and reading skills are highly correlated [4]. Additionally, researches have found that author recognition tasks can be used to assess how much an individual reads [18], which correlates with reading proficiency. Inspired by the above studies, in addition to the generation task, which corresponds to writing, we add a reading comprehension task that aims to improve the model's ability to better understand the style of an author. Specifically, we introduce the author distinction task, which requires the model to decide whether a given pair of documents are written by the same author or not.

Given a document d_{ui} written by a user u, for half of the time, we randomly sample another document d_{uj} from the same user u to form a positive training example $(x, y) = ((d_{ui}, d_{uj}), true)$. Otherwise, we randomly sample another document d_{vk} from another user v $(v \neq u)$ to form a negative example $(x, y) = ((d_{ui}, d_{ok}), false)$. Note that we use text labels $y \in \{true, false\}$ since the generation model $\mathbf{G}$ is a sequence-to-sequence model.

Since the generation model is simultaneously trained on two tasks, personalized generation and author distinction, we prepend a task-level instruction to the model input to help the model distinguish which task to perform. For the personalized generation task, the instruction is "*Finish the passage in the user voice*". While for the author distinction task, the instruction is "*Predict whether two passages are from the same author*".

Note that all the documents for the author distinction task are sampled from **users** that **never** appear in the validation or test data of the personalized generation task.

6 EXPERIMENT SETUP

In this section, we describe our experiment setup, including datasets, training details, competing methods, and metrics.

6.1 Datasets

We evaluate our models on three public datasets, each from a representative domain. A summary of data statistics can be found in Table 1. For all the three datasets, we give their respective definition of a document.

The Avocado Research Email Collection [22] consists of emails and attachments taken from 279 accounts of an information technology company named "Avocado". We follow the processing steps of LaMP [29] and group emails by sender addresses. Note that the number of senders/users can be more than 279 accounts as there are senders outside of the company. We treat an email as a document and its subject as the title.

The Amazon review data [20] provides user reviews from Amazon on different product categories. Since the entire dataset is very large, we choose the biggest category, books, which has the largest number of reviews. We group reviews by users. We treat reviews as documents and review titles as document titles.

The Reddit comments dataset [35] consists of all the posts and comments available on Reddit from 2007 to 2015. We treat both posts and comments as documents and group them by users.

For all the three datasets, we deduplicate identical documents from each user's personal context. A document is qualified to be a *current document*, which is the document to be generated, if this document is longer than 300 characters and the document author has written at least 2 documents before this one. For each qualified current document, we generate an example. We only keep users who have at least 5 examples. We include up to 50 examples per user in case the datasets are dominated by certain active users.

In order to evaluate the model's ability to generalize, we partition the datasets by **users** so that the validation and test sets only contain documents from users that are unseen in the training set. The partition ratio of users in train/validation/test sets are 85/5/10.

6.2 Training details

We finetune the T5-11B [26] model for all the summarization, synthesis, and personalized generation tasks. The T5-11B models are optimized by the Adafactor algorithm [32] with a base learning rate of 0.001. We use the first 1,000 training steps for warmup with a linear warmup scheduler. We additionally apply the square root normalized decay of the learning rate. We train the models until the performance converges on the validation set. Beam search [7] is used for decoding with a beam size of 4.

For multitask learning, we simply mix the examples for personalized generation and author distinction in the ratio of 1:1.

6.3 Competing methods

As mentioned in Section 2, most personalized generation models are proposed for a particular domain, which relies on domain specific knowledge or features. The work closest to ours is LaMP [29], which employs a retrieval based method for personalization using LLMs. Since LaMP focuses on sentence-length generation, it does not explore the advantage of utilizing snippet- or document-based strategies as we have done in this paper.

We consider the following competing methods for better understanding of credit assignment.

Baselines. We consider these baselines.

- ImmedCtx. This method only uses the immediate context x_t as the model input, which is the title and a short start of the current document.
- UserID. We add User ID to the model input and train the model using the next snippet generation task. The user ID helps the model memorize the personal style of each user.

Table 1: Dataset statistics.

	#avg chars of current docs	#avg past docs per current doc	#users			#current docs (#examples)		
			Train	Val.	Test	Train	Val.	Test
Avocado email	3,648.8	42.3	501	27	53	13,305	764	1,227
Amazon review	1,056.7	45.1	354,275	20,789	41,331	5,349,661	311,469	621,438
Reddit	658.5	88.5	240,626	14,223	28,306	3,858,731	231,307	455,582

Since this model performs better on users seen during training, we include all documents that are never used for prediction to train the model, including documents from the validation and the test set. The next snippet generation task requires the model to generate the next snippet given a snippet from a document.
Note that the immediate context includes a short start of the current document, which is also the start of the ground-truth output. Other generation models learn to copy the start to their output to minimize loss but this baseline model is not trained to do so. To make the comparison fair, as postprocess we prepend the start of the current document included in the immediate context to the model output.

- LLMZeroShot. We use the PaLM 2 model [1], which is a new state-of-the-art LLM, for zero-shot generation. PaLM 2 is fed with the input of the *best* configuration based on experiments, including the task instruction to prompt the model to perform the personalized generation task, the immediate context, the context dependent summary, the context dependent synthesis, and the retrieved entries from the best ranking strategy. Similar to the UserID baseline, we also prepend the start of the current document to the model output.

Retrieval augmented methods. We experiment with the ranking strategies introduced in Section 5.2: one sparse retrieval based method RankDocBM25, and three dense retrieval based methods – RankDocDense, RankSnippet, and RankDocBySnpt. In addition, we add a recency based baseline, RecentDoc, which retrieves most recent documents written in the past as ranked entries.

For all these methods, the personalized generation model **G** is trained on the immediate context x_t and the ranked entries $\mathcal{E}_t$.

Summarization. We examine the summarization methods introduced in Section 5.3 by performing summarization on top of *the best configuration of ranking strategies.* The personalized generation model **G** is trained on the immediate context x_t, the output from the summarization model $\mathbf{Su}(x_t, \mathcal{E}_t)$, and the ranked entries $\mathcal{E}_t$. We refer to context independent summarization as SumCtxInd, and context dependent summarization as SumCtx.

Synthesis. We evaluate the synthesis methods introduced in Section 5.4 by applying synthesis on top of *the best configuration of summarization methods.* The personalized generation model **G** is trained on the immediate context x_t, the summary from the summarization model $\mathbf{Su}(x_t, \mathcal{E}_t)$, the synthesis from the synthesis model $\mathbf{Sy}(x_t, \mathcal{E}_t)$, and the ranked entries $\mathcal{E}_t$. We refer to context independent synthesis as SynCtxInd, and context dependent synthesis as SynCtx.

Multitask Training. We use AuthorPred to refer to the multitask setting, where we add the *author distinction* task on top of the best configuration of the single (generation) task to jointly train the generation model.

6.4 Metrics

For each generated current document, we compute its overlap with the ground-truth current document. We adopt Bleu [23], Rouge-1, Rouge-2 and Rouge-L [15] as evaluation metrics, which have been widely used in personalized generation tasks [14, 29].

We conduct statistical significance tests using the paired t-test.

7 EXPERIMENTAL RESULTS

7.1 Overall performance

Table 2: Overall performance(%) on the Avocado email dataset. *, † indicate statistically significant improvement over ImmedCtx, RankDocBM25 respectively at the level of 0.01.

Avocado email	Bleu	Rouge-1	Rouge-2	Rouge-L
Baselines				
ImmedCtx	17.27	32.36	21.45	28.58
UserID	13.28	32.33	20.95	27.86
LLMZeroShot	14.93	35.06*	22.11*	28.52
Retrieval augmented methods				
RecentDoc	19.57*	35.64*	23.96*	31.25*
RankDocBM25	21.19*	37.69*	25.99*	33.07*
RankDocDense	19.43*	35.62*	23.71*	30.90*
RankSnippet	18.69*	35.82*	23.26*	30.78*
RankDocBySnpt	21.06*	37.42*	25.65*	32.90*
+Summarization				
SumCtxInd	21.23*	37.58*	25.79*	33.15*
SumCtx	23.17*†	39.31*†	26.64*†	34.37*†
+Synthesis				
SynCtxInd	23.06*†	39.24*†	26.72*†	34.52*†
SynCtx	**23.44*†**	40.38*†	26.93*†	34.34*†
+Multitask				
AuthorPred	23.27*†	**41.02*†**	**28.60*†**	**35.70*†**

The overall performance on the three datasets are listed in Table 2, 3, and 4 respectively. In general, retrieval augmented methods perform better than baselines that do not utilize the retrieved information. Summarization and synthesis bring additional gains when they are dependent on the immediate context. Multitask learning

Table 3: Overall performance(%) on the Amazon review dataset. *, † indicate statistically significant improvement over ImmedCtx, RankDocBySnpt respectively at the level of 0.01.

Amazon review	Bleu	Rouge-1	Rouge-2	Rouge-L
Baselines				
ImmedCtx	17.83	36.22	22.49	31.77
UserID	17.61	36.62	22.85	32.11*
LLMZeroShot	16.29	38.74*	22.52	30.79
Retrieval augmented methods				
RecentDoc	19.09*	37.51*	23.37*	32.67*
RankDocBM25	19.49*	37.79*	23.56*	32.85*
RankDocDense	19.38*	37.49*	22.92*	32.48*
RankSnippet	19.44*	37.45*	23.10*	32.50*
RankDocBySnpt	19.35*	38.28*	23.87*	33.23*
+Summarization				
SumCtxInd	19.17*	38.33*	23.66*	33.35*
SumCtx	19.81*†	38.66*	24.25*†	33.57*
+Synthesis				
SynCtxInd	19.84*†	38.68*	24.31*†	33.61*
SynCtx	**19.87*†**	**39.46*†**	**24.66*†**	**33.97*†**
+Multitask				
AuthorPred	19.78*†	39.36*†	24.56*†	33.87*†

further improves the model's generation ability in most datasets. We compare important methods in detail below.

Comparison among baselines. Comparing with ImmedCtx, UserID performs surprisingly well, especially on the Amazon review and the Reddit data. This is because the UserID model is trained on the next snippet prediction task, resulting in relatively shorter generated output. By memorizing the user IDs, the model has an advantage on datasets with shorter document length, e.g., the Amazon review and the Reddit data. However, since this model requires user IDs as input, it has problems in generalizing to new users or scaling to a large number of users.

LLMZeroShot is provided with the same input as SynCtx. The reduced performance suggests that finetuning is critical to the personalized generation task.

Retrieval augmented methods. All the retrieval augmented methods outperform ImmedCtx, which excludes all documents from a user's personal context. This indicates that past documents provide useful information when the model generates the current document for a user.

RecentDoc unexpectedly performs on par with some similarity based retrieval methods in many cases, especially on the Avocado email dataset. But it still performs worse than RankDocBySnpt. On one hand, RecentDoc is a decent strategy as a user's recent documents might share similar writing style, and even similar content, e.g., a user is writing multiple emails concerning a particular topic, or a user starts to take interest in a particular book genre and are thus writing similar book reviews. On the other hand, providing information more relevant to the current topic based on the immediate context using RankDocBySnpt instead of simply retrieving the most recent content can further improve the generation performance.

RankDocBM25 performs closely to RankDocBySnpt except for the Amazon review dataset, where RankDocBM25 is less effective. This suggests that BM25 is still a powerful retrieval strategy even when the query, which is the immediate context, is longer than common search queries and is written in natural language.

RankDocBySnpt consistently performs better than or equally well as other retrieval based methods by combining the strength of RankDocDense and RankSnippet. By retrieving on the snippet level, dense retrieval becomes more effective by encoding less information in a single embedding. By ranking documents that contain the retrieved snippets, the generation model is able to extract more diverse or detailed information for content generation. RankDocDense, RankSnippet and RankDocBySnpt all perform well on the Reddit data, due to the relatively short document length of this dataset.

Summarization. SumCtxInd performs similarly as retrieval augmented methods without the summarization step, while SumCtx outperforms retrieval augmented methods. This indicates that a generic summary does not provide additional information to the generation model. The summary is more useful when it considers the immediate context. It guides the generation model to the incorporation of possible topics or phrases when generating the

Table 4: Overall performance(%) on the Reddit dataset. *, † indicate statistically significant improvement over UserID, RankSnippet respectively at the level of 0.01.

Reddit	Bleu	Rouge-1	Rouge-2	Rouge-L
Baselines				
ImmedCtx	26.33	43.69	33.52	40.53
UserID	26.55	44.71	34.61	41.53
LLMZeroShot	24.04	43.67	31.24	37.85
Retrieval augmented methods				
RecentDoc	28.23*	44.83	34.57	41.72
RankDocBM25	28.97*	45.71*	35.13*	42.52*
RankDocDense	28.82*	45.61*	35.22*	42.56*
RankSnippet	29.08*	45.94*	35.39*	42.68*
RankDocBySnpt	28.87*	45.67*	35.24*	42.54*
+Summarization				
SumCtxInd	28.91*	45.71*	35.26*	42.57*
SumCtx	28.92*	46.09*	35.82*†	43.14*†
+Synthesis				
SynCtxInd	28.82*	45.91*	35.87*†	43.09*†
SynCtx	**29.19***	46.45*†	36.13*†	43.27*†
+Multitask				
AuthorPred	29.13*	**47.11*†**	**36.94*†**	**43.95*†**

current document. Without the context dependent summary, the generation model needs to consider the high-level topics and the exact words to generate at the same time, making the task harder.

Synthesis. We observe patterns similar as summarization – SynCtxInd performs on par with SumCtx, a method without synthesis, while SynCtx outperforms SumCtx. This means that synthesis is useful only when it is dependent on the immediate context. We also see that SynCtx improves the Rouge-1 metric more than other metrics. This is due to our design choice of the synthesis stage by identifying important unigrams. More sophisticated synthesis methods are worth exploring as future work to improve metrics other than Rouge-1.

Multitask. By jointly learning to predict whether two documents are from the same author, AuthorPred performs better than SynCtx, which has a single task setting, on the Avocado email and the Reddit datasets, and performs closely on the Amazon review dataset. This indicates that improving the model's reading ability by differentiating the writing style of different authors does no harm to the generation performance, which often benefits.

7.2 Formulation of input and output

Since Table 2, 3, and 4 have already included the ablation study by reporting the performance of adding one new component at a time, we additionally investigate whether varying the formulation of the input and output will lead to significant change in performance. We study the following settings.

- NoRankedEntries. We investigate whether the ranked entries are still necessary when the summary and the synthesized outcome are available. To this end, we train the generation model using the immediate context, the context dependent summary, and the context dependent synthesis.
- RemoveDocStart. The immediate context includes a short start of the current document. For all the previous experiments, this short start is still present in the ground-truth label, which is the current document, when we train the generation model. We study the effect of removing the short start from the ground-truth label. Note that to make the comparison fair, we prepend the short start to the model generated text when computing metrics.
- ImmedCtxAtEnd. In all the experiments above, the information sources are placed in the order of the immediate context, the summary, the synthesis, and the ranked entries. We study whether the order matters by placing the immediate context at the end.

Table 5: Analysis of formulation of input and output on the Avocado email dataset based on absolute change of performance(%) over AuthorPred. Other datasets show similar patterns and are omitted to save space.

Avocado email	Bleu	Rouge-1	Rouge-2	Rouge-L
NoRankedEntries	-2.53	-2.6	-3	-2.75
RemoveDocStart	-0.85	-0.74	-2.25	-1.78
ImmedCtxAtEnd	-0.12	0.05	0.13	-0.08

Table 6: Examples of model output. Italic words are used in the immediate context.

Ground-truth doc: *I just did not enjoy this book. I discovered this author a few weeks ago and have been making my way through her* cocktail series but this one was not good for me. Didnt particularly like the characters (and I get it, Lucas looks like Prince Harry) and the plot was slow and less than engaging. I finished it, but only because Im stubborn.
SumCtx output: *I just did not enjoy this book. I discovered this author a few weeks ago and have been making my way through her* books. I really liked the first book in the series, but this one was just not my favorite. I just didn't find myself connecting with the characters. I found the heroine to be a bit too naive and I just didn't find myself falling for the hero.
RankDocBySnpt output: *I just did not enjoy this book. I discovered this author a few weeks ago and have been making my way through her* books. I've enjoyed the first two books in this series and I'm not sure why I didn't like this one. I'm not sure if it's because I'm not a fan of the whole "friends to lovers" thing, but I just didn't like the way this book was written.
Why SumCtx is better: the summarization model mentions *I didn't fall for the hero*, which makes the generation model comment more on the characters.
Ground-truth doc: *Thanks for your insight. I do websites, mobile applications, desktop applications. I have a lot of success with odesk and have been using it ever* since. I have been thinking of developing stuff for my own the way I do it for my clients. I just get too busy with side projects and my full-time employment.
SynCtx output: *Thanks for your insight. I do websites, mobile applications, desktop applications. I have a lot of success with odesk and have been using it ever* since. I have a lot of clients and I have been using it for a long time. I am a professional developer though and I have been doing this for a long time.
SumCtx output: *Thanks for your insight. I do websites, mobile applications, desktop applications. I have a lot of success with odesk and have been using it ever* since. I have a lot of projects. I have a lot of experience with php, javascript, html, css, javascript, html5, c#, javascript, c#
Why SynCtx is better: the synthesis model mentions *developer* and *freelancer* so that the generation model can focus on the topic of development and clients.
Ground-truth doc: *As another reviewer wrote, this book flows gently. There isn't a lot of action and yes, the timeline bounces around but it's* not hard to figure out what is happening. But fair warning - this is a sad story. I got choked up once or twice. It's very well written but the story isn't really groundbreaking.
AuthorPred output: *As another reviewer wrote, this book flows gently. There isn't a lot of action and yes, the timeline bounces around but it's* not confusing. The characters are well-developed and the story is interesting. I did find the ending a little sad, but I guess that's what I was expecting.
SynCtx output: *As another reviewer wrote, this book flows gently. There isn't a lot of action and yes, the timeline bounces around but it's* not confusing. The story is about a group of friends who have been friends since childhood. They are all in their late 30s and have been friends since childhood.
Why AuthorPred is better: The author distinction task helps the model understand that this user often gives a high-level review of the story instead of going to details.

Due to the space limit, we only show the performance on the Avocado email data in Table 5. Other datasets show similar patterns.

NoRankedEntries underperforms AuthorPred by a large margin, meaning that the generation model still relies on the retrieved

entries for information, e.g., word usage and writing style, even when the summary and the synthesis are available. There is a performance degradation of RemoveDocStart. We suspect that the presence of the short start of the current document in both the input and the ground-truth label helps the model understand the task better. This is supported by the observation that the model converges faster during training when the short start is included in the ground-truth label. The close performance between ImmedCtxAtEnd and AuthorPred indicates that reordering the information sources does not affect the performance at least in the case of finetuning.

7.3 Case studies

We provide some illustrative examples in Table 6 for a better understanding of why the proposed method could provide better guidance on how to generate personalized content.

8 CONCLUSION

We propose a general approach for teaching large language models for personalized text generation. Analogous to how students are instructed to write from sources in a sequence of steps, the proposed approach consists of multiple stages: retrieval, ranking, summarization, synthesis, and generation. Additionally, inspired by the observation that reading and writing skills are correlated, we create a multitask setting that improves the model's reading ability by distinguishing the authorship of given document pairs. This multitask setting further improves the model's ability to generate personalized text empirically. We evaluate our models on three publicly released datasets from representative domains. Our results demonstrate the effectiveness of the multistage multitask framework. Investigation into the incorporation of world knowledge, e.g., product information, is a promising direction for future work.

REFERENCES

[1] Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403* (2023).

[2] James P. Callan. 1994. Passage-Level Evidence in Document Retrieval. In *SIGIR '94*, Bruce W. Croft and C. J. van Rijsbergen (Eds.). Springer London, London, 302–310.

[3] Adeline Cooney, Eamon Darcy, and Denis Casey. 2018. Integrating reading and writing: supporting students' writing from source. *Journal of University Teaching & Learning Practice* 15, 5 (2018), 3.

[4] Marion Crowhurst. 1990. Reading/writing relationships: An intervention study. *Canadian Journal of Education/Revue canadienne de l'éducation* 15, 2 (1990), 155–172.

[5] Sumanth Dathathri, Andrea Madotto, Janice Lan, Jane Hung, Eric Frank, Piero Molino, Jason Yosinski, and Rosanne Liu. [n.d.]. Plug and Play Language Models: A Simple Approach to Controlled Text Generation. In *International Conference on Learning Representations*.

[6] Christiane Fellbaum. 1998. *WordNet: An electronic lexical database*. MIT press.

[7] Markus Freitag and Yaser Al-Onaizan. 2017. Beam Search Strategies for Neural Machine Translation. *ACL 2017* (2017), 56.

[8] Yong-Bing GAO, Jun-Tian GAO, Rong MA, and Li-Dong YANG. [n.d.]. Research on user granularity-level personalized social text generation technology. *Journal of Computer Applications* ([n. d.]), 0.

[9] Misha Khalman, Yao Zhao, and Mohammad Saleh. 2021. ForumSum: A multi-speaker conversation summarization dataset. In *Findings of the Association for Computational Linguistics: EMNLP 2021*. 4592–4599.

[10] Byeongchang Kim, Hyunwoo Kim, and Gunhee Kim. 2019. Abstractive Summarization of Reddit Posts with Multi-level Memory Networks. In *NAACL-HLT*.

[11] Ben Krause, Akhilesh Deepak Gotmare, Bryan McCann, Nitish Shirish Keskar, Shafiq Joty, Richard Socher, and Nazneen Fatema Rajani. 2021. GeDi: Generative Discriminator Guided Sequence Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*. 4929–4952.

[12] Juncen Li, Robin Jia, He He, and Percy Liang. 2018. Delete, Retrieve, Generate: a Simple Approach to Sentiment and Style Transfer. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. 1865–1874.

[13] Junyi Li, Siqing Li, Wayne Xin Zhao, Gaole He, Zhicheng Wei, Nicholas Jing Yuan, and Ji-Rong Wen. 2020. Knowledge-enhanced personalized review generation with capsule graph neural network. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 735–744.

[14] Pan Li and Alexander Tuzhilin. 2019. Towards Controllable and Personalized Review Generation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 3237–3245.

[15] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.

[16] Alisa Liu, Maarten Sap, Ximing Lu, Swabha Swayamdipta, Chandra Bhagavatula, Noah A Smith, and Yejin Choi. 2021. DExperts: Decoding-Time Controlled Text Generation with Experts and Anti-Experts. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 6691–6706.

[17] Pierre-Emmanuel Mazare, Samuel Humeau, Martin Raison, and Antoine Bordes. 2018. Training Millions of Personalized Dialogue Agents. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2775–2779.

[18] Sean Patrick McCarron and Victor Kuperman. 2021. Is the author recognition test a useful metric for native and non-native English speakers? An item response theory analysis. *Behavior Research Methods* 53, 5 (2021), 2226–2237.

[19] Jianmo Ni, Gustavo Hernandez Abrego, Noah Constant, Ji Ma, Keith Hall, Daniel Cer, and Yinfei Yang. 2022. Sentence-T5: Scalable Sentence Encoders from Pre-trained Text-to-Text Models. In *Findings of the Association for Computational Linguistics: ACL 2022*. 1864–1874.

[20] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 188–197.

[21] Jianmo Ni, Chen Qu, Jing Lu, Zhuyun Dai, Gustavo Hernández Ábrego, Ji Ma, Vincent Y Zhao, Yi Luan, Keith B Hall, Ming-Wei Chang, et al. 2021. Large dual encoders are generalizable retrievers. *arXiv preprint arXiv:2112.07899* (2021).

[22] Douglas Oard, William Webber, David Kirsch, and Sergey Golitsynskiy. 2015. Avocado research email collection. *Philadelphia: Linguistic Data Consortium* (2015).

[23] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.

[24] Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 1532–1543.

[25] Hongjin Qian, Xiaohe Li, Hanxun Zhong, Yu Guo, Yueyuan Ma, Yutao Zhu, Zhanliang Liu, Zhicheng Dou, and Ji-Rong Wen. 2021. Pchatbot: A large-scale dataset for personalized chatbot. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2470–2477.

[26] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research* 21, 1 (2020), 5485–5551.

[27] Sudha Rao and Joel Tetreault. 2018. Dear Sir or Madam, May I Introduce the GYAFC Dataset: Corpus, Benchmarks and Metrics for Formality Style Transfer. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. 129–140.

[28] Stephen E Robertson, Steve Walker, Susan Jones, Micheline M Hancock-Beaulieu, Mike Gatford, et al. 1995. Okapi at TREC-3. *Nist Special Publication Sp* 109 (1995), 109.

[29] Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. 2023. LaMP: When Large Language Models Meet Personalization. *arXiv preprint arXiv:2304.11406* (2023).

[30] Abigail See, Peter J Liu, and Christopher D Manning. 2017. Get To The Point: Summarization with Pointer-Generator Networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1073–1083.

[31] Timothy Shanahan. 2015. Common Core State Standards: A new role for writing. *The Elementary School Journal* 115, 4 (2015), 464–479.

[32] Noam Shazeer and Mitchell Stern. 2018. Adafactor: Adaptive learning rates with sublinear memory cost. In *International Conference on Machine Learning*. PMLR, 4596–4604.

[33] Tianxiao Shen, Tao Lei, Regina Barzilay, and Tommi Jaakkola. 2017. Style transfer from non-parallel text by cross-alignment. *Advances in neural information processing systems* 30 (2017).

[34] Khrystyna Skopyk, Artem Chernodub, and Vipul Raheja. [n.d.]. Personalizing Large Language Models. ([n. d.]).

[35] Stuck_In_the_Matrix. 2015. Reddit Public Comments (2007-10 through 2015-05). (2015). https://www.reddit.com/r/datasets/comments/3bxlg7/i_have_every_publicly_available_reddit_comment/

[36] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).

[37] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[38] Yuwei Wu, Xuezhe Ma, and Diyi Yang. 2021. Personalized response generation via generative split memory network. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 1956–1970.

[39] Kevin Yang and Dan Klein. 2021. FUDGE: Controlled Text Generation With Future Discriminators. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 3511–3535.

[40] Chenhan Yuan and Yi-chin Huang. 2020. Personalized sentence generation using generative adversarial networks with author-specific word usage. *Computación y Sistemas* 24, 1 (2020), 17–28.

[41] Saizheng Zhang, Emily Dinan, Jack Urbanek, Arthur Szlam, Douwe Kiela, and Jason Weston. 2018. Personalizing Dialogue Agents: I have a dog, do you have pets too?. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2204–2213.

[42] Zhirui Zhang, Shuo Ren, Shujie Liu, Jianyong Wang, Peng Chen, Mu Li, Ming Zhou, and Enhong Chen. 2018. Style transfer as unsupervised machine translation. *arXiv preprint arXiv:1808.07894* (2018).

[43] Hanxun Zhong, Zhicheng Dou, Yutao Zhu, Hongjin Qian, and Ji-Rong Wen. 2022. Less is More: Learning to Refine Dialogue History for Personalized Dialogue Generation. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 5808–5820.